

## Data science Assignments

In this assignment, you will build a small data preprocessing pipeline using Titanic dataset.

The Goal is not just to clean the data.

You should think like a developer building a preprocessing component that could later become part of real MLOPs project.

Your Solution should be :

- Modular
- Reusable
- Configuration- driven
- Easy to Understand
- Able to handle unexpected situations

You will implement the pipeline using Python and pandas.

### **1- Read the Dataset**

Create the following function

```
Read_data_file(file_path)
```

The function should read the Titanic CSV file and return a panda DataFrame.

Make sure the function can deal with common problems such as:

- The file does not exist.
- The file cannot be read.
- The provided path is invalid.

You do not need to overcomplicate the error handling, but don't assume that the file will always be perfect.

Think about it

What should your function do if the file path is wrong?

Should the program crash with long pandas' error, or should the function provide a more useful message?

### **2- Remove Unnecessary Features**

Create :

```
Drop_unnecessary_features(df, cols_to_drop)
```

The function should remove the columns provided through `cols_to_drop`

### **Important**

Don't write something like this inside the function:

```
cols_to_drop = ["PassengerId", "Name", "Ticket"]
```

the function should not know anything specifically about the Titanic dataset.  
Instead, the columns should be come from the configuration.

For example, you may have:

```
Config/  
    __init__.py  
Config.py
```

Your configuration can contain the columns that should be removed.

The function should simply receive the list and perform the operation.

Why?

Imagine that tomorrow you use the same function on another dataset.

You should be able to change the configuration without rewriting the function

Inspect the Dataset

Create :

```
Check_data_type(df)
```

This function should help you understand the structure of the database.

For every column , show at least :

- Column name
- Datatype
- Number of unique values

Return the result as a transpose Data frame that easy to read.

For example, your output should allow you to quickly answer the questions such as :

- What the is Age?
- How many unique values does Embark have?
- Which columns look like categories feature?

Don't just print `df.dtypes`

The idea is to create a small data-quality report.

### **3- Convert categorical Features**

#### **Create**

```
Handle_and_convert_dtype(df, categorical_col)
```

The function should convert the columns supplies in categorical to pandas :

Category

Again, don't hardcode the list inside the function.

Avoid

```
df['sex'] = df['sex'].astype("category")  
df['Embark'] = df['Embark'].astype(category)
```

Instead, the function should work with whatever list is passed to it.

For Example :

```
Categorical_cols -> ["sex", "Embarked"]
```

The exact list should come from your configuration.

Think about this

What happens if your configuration contains a column that does not exist in the Data Frame?

Your code should have a reasonable way of dealing with this situation.

### **4- Handle Missing Values**

Now come the most important part of the assignment.

Create:

```
Handle_null_values(df)
```

This function should apply the following business rules.

#### **Rule -1 Embarked**

If Embarked contains missing values:

Drop those rows.

Do not use : fillna()

To replace them with the most common value.

Rule 2 – cabin

Cabin contains a large amount of missing data

For this assessment

Drop the entire cabin column.

Do not try to fill the missing values.

“Write here the reason for doing that.....”

.....  
.....  
.....

### Rule -3

This where you need to think carefully.

You are not allowed to fill missing Age values using the overall median, so this is Not acceptable.

```
df['Age'] = df['Age'].fillna(df['Age'].median())
```

Instead, the missing age should be estimated using the passengers”

Pclass

Sex

In other words. Calculate the median age for each:

Pclass + sex

Combination

Think of it like this:

| <u>Pclass</u> | Sex    | Median Age |
|---------------|--------|------------|
| 1             | female | ...        |
| 1             | male   | ...        |
| 2             | female | ...        |
| 2             | male   | ...        |
| 3             | female | ...        |
| 3             | male   | ...        |

Then use the appropriate group median when filling a missing Age.

Example

If a passenger belongs to:

Pclass = 2

Sex = female

Their missing should be filled using the median age of other passengers belonging to the same group.

One More thing to think About

What if one of your groups does not have enough valid Age values to calculate a median?

For example, imagine that a (pclass, sex) group contains only missing ages.

Your code should have a reasonable strategy for this situation.

There is no single magic answer here.

What matters is that:

- You recognize the problem
- You choose a sensible solution
- You explain your decision in your code /comments

## 5- Configuration

Your preprocessing function should not contain dataset-specific configuration at minimum, move these outside the functions:

Columns to drop

Categorical columns

A possible structure is :

```
project/
├── config/
│   ├── __init__.py
│   └── config.py
├── data/
│   └── raw/
│       └── titanic.csv
└── preprocessing.py

└── Main.py
```

The exact project structure is up to you.

The important idea is :

Configuration ---→ preprocessing functions → processing dataset

## **6- Put Everything Together**

After creating your five functions, run them as a pipeline.

A possible flow is:

Read Data -> Drop unnecessary features -> inspect Data->convert Data types->  
handle missing values-> final Dataset

Errors.

Run All

You should not need to manually execute cell in a particular order.

Once you have called the functions, do not stop.

To prove that the preprocessing was successful, add a couple of checks.

For example, confirm that:

The columns that weren't needed were removed.

The cabin was taken removed.

In Embarked, there are no values missing.

Age values that were missing were updated.

Dtype category is present in the categorized columns.

Above all, show that Age was entered via the:

Pclass × Sex

grouping as opposed to the global median.

The Last Task

Imagine that your instructor modifies the setup before submitting.

For instance:

Today:

drop →="PassengerId", "Ticket"]

Tomorrow:

`drop →="PassengerId", "Ticket", "Name"]`

Your functions ought to keep operating.

Only the configuration should need to be updated.

Opening the function and rewriting its logic shouldn't be necessary.

The category columns follow the same principle.

## **Common Mistakes**

Stay clear of the following:

- Columns to be dropped inside the function are hardcoded.
- The function's category columns are hardcoded.
  - utilizing the age global median.
  - Rather than dropping the impacted rows, filling embarked.
  - attempting to impute it while keeping the cabin.
  - combining all the preparation steps into a single, large function.
  - writing code that is limited to the dataset you tested.
  - Ignoring unexpected or absent columns without considering how

For this structure kindly apply the above all function inside the preprocessing.py file and you can create a main.py file to run the pipeline for read the file and user can input the to remove unnecessary feature saved on config.py also user can check for datatype and all function make user to choice very available functions

Goodluck

George Samuel

01022331830

SagaGen-Academy  
George Samuel Mse, PhD Researcher – Cairo University  
01022331830

SagaGen-Academy-George Samuel